

LightWave–RenderMan archives: value for Blender material conversion

Assessment date: 2026-09-10. Converter baseline: lwconvert 0.3.0.

Assessment

Both archives contain readable RenderMan Shading Language (RSL). LightMan is a substantial research reference; Light-R is a smaller reference for basic shading and lighting. The strongest findings are LightWave texture projections, procedural texture approximations, and embedded material-generation formulas. They can guide a richer native material representation and Blender node generation, but do not provide a complete, directly reusable LightWave renderer.

Recommendation: use the findings to develop and validate material translation within the existing **C** → **IR** → **Blender** architecture. Keep ordinary Blender nodes as the default output. Neither historic plugin should become a conversion dependency, and their redistribution terms require attention before incorporating code.

1. What is actually present

Archive	Contents relevant to this assessment	Value
lighter_0_30.zip	Light-R 0.30 alpha, February 2005, by Felipe Esquivel. Seven files: Windows x86 lighter.p , five .sl sources, README. Targets LightWave 7/8.	Basic surface model and light falloff; no texture system.
lightman11_lw7.zip	LightMan 1.1, by Timm Dapper. 53 files plus seven directory entries: Windows x86 lightman.p , eight .sl files, LightMan/lwshad.h (976 lines), HTML manual, seven renderer configurations, two LWO/LWS example pairs and a TIFF. Manual dated May 2002; some configuration entries dated March 2003.	Projections, procedural kernels, material-generation fragments and useful test inputs.

Both ZIPs pass CRC verification. Inspection covered text sources, documentation, printable binary strings and example object chunks. Neither plugin was executed; no shader compilation or comparison against original LightWave renders was performed. Neither archive supplies the exporter's C/C++ source or build system. The [.h](#) file contains **RSL**, despite its C-like extension; [.rmr](#) files configure renderer commands and capabilities rather than implement materials.

2. LightMan: the useful material reference

Readable shader code

The principal source is [LightMan/lwshad.h](#). Line references below refer to the original archive member.

Feature	Evidence	Implication for our converter
Planar, cylindrical, spherical and	lwplanarprojection , lwcylindricalprojection , lwsphericalprojection , lwcubicprojection ,	Strong starting point for generated UVs or Blender coordinate node groups. Validate orientation, seams,

Feature	Evidence	Implication for our converter
cubic mapping	lines 188–275. Axis-dependent signs, half-unit offsets, angular wrapping and dominant-normal axis selection are explicit.	cube-axis ties and geometric normals against source examples.
Texture transforms	<code>lwtexcoord</code> , lines 277–294: inverse translation, rotations and reciprocal scale.	Helps recover texture-space conventions. The normal and <code>RDRIVE</code> branches use different rotation-axis assignments: do not copy the transform blindly. Coordinate conversion and parameter ordering need independent checks.
Procedural textures	Nontrivial implementations of Brick, Bump Array, Checkerboard, Crumple, Dots, Fractal Noise, Grid, Honeycomb, Ripples, Turbulence, Veins, Wood and fBm, lines 301–974. Includes Worley/Voronoi helpers and signed-noise macros.	Provides parameter roles, octave accumulation, contrast adjustments and pattern construction, beyond simply substituting similarly named Blender nodes.
Filtering	Brick, Checkerboard and Grid use projected sample widths and fade toward an average at small scales.	Filtering affects the appearance; matching only the pattern equation will not reproduce distant or grazing-angle textures.
Light falloff and fog	<code>lwpointlight.sl</code> , <code>lwspotlight.sl</code> , <code>lwarealight.sl</code> , <code>lwlinearlight.sl</code> , <code>lwfog.sl</code> .	Secondary reference for scene appearance, including clamped linear and inverse-distance falloff, and legacy fog curves. These are not complete light-export implementations.

Material formulas inside the plugin

The general surface shader is generated by the exporter. It is **not supplied as a complete standalone .sl file**, but `LightMan/lightman.p` contains readable RSL generation fragments. These are useful evidence, not recovered generator source or proof that every branch executes.

Binary file offset	Recovered behavior	Interpretation
<code>0x000CFB08</code>	<code>Kglos = pow(2, 10 * Kglos + 2)</code>	LightMan maps its glossiness input to a highlight exponent. For input 0, 0.5 and 1, the exponent is 4, 128 and 4096. Preserve this as a LightMan convention , not a proven universal LightWave-to-Blender mapping.
<code>0x000CF7E4</code> , <code>0x000CF820</code>	Highlight proportional to <code>max(0, N·H)^Kglos</code> , with <code>H = normalize(Ln + V)</code>	A Blinn-style half-vector highlight. A Blender GGX roughness requires fitting or an explicit approximation, rather than copying the exponent into a roughness socket.
<code>0x000CFA20</code> , <code>0x000CF680</code>	Specular tint interpolates between white and surface color using	Preserve Color Highlights, specularity and reflection separately. Reflectivity does not

Binary file offset	Recovered behavior	Interpretation
	<code>colorhighlights</code> ; reflection is scaled independently.	establish metallicity.
<code>0x000CF728– 0x000CF7BC, 0x000CFB48</code>	Separate transparency/refraction and derivative-based bump fragments.	Useful evidence for channel separation, opacity versus transmission, bump sign and scale; insufficient to establish complete evaluation semantics.
<code>0x000D0364– 0x000D09D0</code>	Gradient interpolation, previous-layer luminance, slope, incidence and distance inputs.	Helps design typed gradient inputs and dependencies. Some formulas need scrutiny: the incidence fragment uses <code>90 * dot(N,V)</code> , not an inverse-cosine angle.

Additional strings show layer opacity, inversion, blending and coordinate generation. Their order in the executable is **not shader execution order**. Complete layer dispatch, defaults, serialized parameter decoding and caller-side adjustments remain unavailable as source. The header alone is therefore insufficient to reproduce the material generator.

Limits and misleading feature claims

The manual's §2.2 explicitly describes procedural results as approximate and flags differences in multiply/divide blending. Its older §1.3 still claims only basic surface parameters are exported; §2.2 describes the newer 1.1 texture conversion. Prefer the actual code where these sections disagree.

- **Six procedural functions are no-ops:** `lwcrust`, `lwmarble`, `lwsmoky1`, `lwsmoky2`, `lwsmoky3`, `lwunderwater` only multiply opacity by one. Crust is nevertheless listed as supported in §2.2.
- RenderMan `noise()`/`cellnoise()` supply the underlying noise. These sources do not recover LightWave's original noise implementation or establish equivalence with Blender's noise.
- Ripples hardcodes a factor of **30** in its time term (line 740); time units and frame-rate dependence need verification. Other kernels contain empirical adjustments or unused parameters.
- The manual excludes front projection and surface shading plugins. A binary NDC-projection fragment does not overturn that documented limitation. Weight-map gradients explicitly have an unimplemented fallback in the binary.
- The archive is centered on LightWave 6.5b/7. Matching a procedural name does not establish identical parameters or behavior in older LWOB objects.

Relevance to `content/`

The existing [dataset inventory](#) contains 300 LWO2 procedural blocks. **267 have names corresponding to nontrivial LightMan implementations:** Turbulence 190, Crumple 44, Fractal Noise 16, Veins 11 and Wood 6. Another five are Value, whose unchanged-opacity behavior is identified in the binary. The remaining 28 are heteroTerrain 20, coriolis 4 and Crust 4: absent implementations or a stub.

This is **name overlap, not a fidelity or supported-material percentage**; occurrences can include disabled layers. It makes Turbulence, Crumple and Fractal Noise worthwhile research priorities. Examples include `content/butterfly-tank/01.lwo` and `content/circus/meshes/cindy.lwo`.

The same inventory records 423 LWO2 image-map blocks and 442 gradients. Legacy LWOB contains 241 planar, 55 cylindrical, 51 cubic and 24 spherical image-map declarations, plus Ripples and Underwater. Thus projection work has broad relevance, while **Underwater remains unsolved**. There are also 43 front-projection declarations

across both formats. These archives do not supply implementations of the corpus's SuperCelShader, FastFresnel, BRDF, FPrime or other third-party surface plugins.

Included examples

`LightMan/examples/Objects/texttest.lwo` contains three planar color-image layers across two surfaces, using Y/Z axes and `Images/grid2.tif`. `texttest_c.lwo` contains Dots and Grid color procedurals. Each has a corresponding LWS scene. They are useful local inputs for texture orientation and procedural tests, but **do not constitute reference renders or a comprehensive regression suite**.

3. Light-R: useful, but narrower

`LR_surface.sl` implements diffuse and ambient contributions, luminosity, a specular highlight, back-side diffuse translucency, reflection, opacity and a refraction path. It also exposes color/diffuse/specular/reflection outputs that suggest useful diagnostic render passes.

Its parameter named `roughness` is passed directly to RSL `phong()` (line 37); it must not be treated as Blender roughness. The source also warrants caution: `doubleSide == 1` suppresses the back side; the refraction branch computes Fresnel coefficients but does not use them to weight the traced color (lines 65–70). The binary exporter supplies parameter values, so shader defaults alone do not describe exported LightWave behavior.

The three light shaders document legacy attenuation modes. Unlike LightMan, Light-R's linear expression is not clamped at zero. Its README lists textures and averaged normals as unfinished. There is **no image-projection, layered-texture, gradient or procedural material implementation** here. Use it to cross-check basic shading concepts, not as the main material-conversion reference.

4. How to apply the findings

The current `material structure`, `LWO parser` and `glTF writer` preserve/expose basic scalars but do not evaluate texture layers. glTF currently uses scalar color/diffuse/emission/opacity with fixed roughness and no texture bindings. The next prerequisite is richer material data in the IR.

Recommended sequence:

1. **Expand the C parser and IR:** glossiness, reflection, IOR, translucency, color highlights, bump, ordered texture layers, projections/transforms, images, gradients and envelopes. Preserve raw values and unsupported payloads alongside interpretation metadata.
2. **Implement image mapping first:** explicit UV bindings, then static planar/cylindrical/spherical/cubic projections. Use independently created axis-marked textures and cases covering scale, rotation, reference objects, wrapping and seams. Account for the existing glTF UV flip exactly once.
3. **Develop procedural approximations:** start with Turbulence, Crumple and Fractal Noise. Compare controlled parameter sweeps against LightWave renders when available; label unvalidated behavior as approximate. Do not mark a stub as supported.
4. **Build Blender materials from IR:** generate ordinary coordinate, math, texture, ramp and shader nodes using a Python adapter launched through `blender --background --python ...`. Blender's installed 4.2 command-line help confirms both options. No maintained addon or GUI interaction is required; the adapter itself remains versioned conversion code.
5. **Validate shading separately:** use controlled illumination, color management and scalar/channel passes for glossiness, transparency, bump and emission. Keep Blender's richer graph separate from OBJ/glTF approximations; bake supported channel results where appropriate, without discarding the native graph.

RSL cannot simply be loaded into Blender's Script node. Blender uses OSL for that node, and OSL materials use renderer closures rather than RSL's explicit light loop. In Blender 4.2 this is Cycles-only, with CPU/OptiX restrictions; `trace()` also does not provide RSL-style shaded reflection lookup. An OSL port may be an optional research route, but is not a shortcut to exact LightWave shading or portable default output. [Blender 4.2 OSL documentation](#).

5. Source reuse and provenance

LightMan is not an unrestricted source release. [LightMan/doc/index.html](#), §8, lines 1758–1763 prohibits redistribution of LightMan or its parts. It permits licensees to use/alter supplied or generated shader sources and headers on condition that changes are contributed back to the author. This assessment does not establish our licensee status or permission to redistribute ports. Establish applicable permission before copying, translating or distributing those sources or examples in the converter.

Light-R's terms are incomplete for source reuse. Its README says “Use this program freely” and attributes modified light shaders to Aqsis, but supplies no explicit source redistribution license. That statement alone does not establish permission to incorporate all shader code. Preserve attribution and investigate upstream terms before reuse.

For now, retain these as research references and pursue independent implementations grounded in format specifications and measured behavior. Inspection copies remain under ignored [build/archive-research/](#); no archival source has been copied into converter code.

Archive SHA-256 values identify the exact versions assessed; binary offsets above are file offsets within this LightMan build:

File	SHA-256
extern/lighter_0_30.zip	6e13d4c7508574bda2a671e29e037dcb9674e2d24e2de07069450da7d4d0ca95
extern/lightman11_lw7.zip	b98a0c442340c1245c613a167a768edf43455c1b399f26cfe67caff3169deb71
LightMan/lightman.p inside the latter archive	4c004d85fa22fb0edc23f55a10b194d3582ba8b01bb92cd20918162930c410c0

The archives materially reduce uncertainty around projections and several procedural patterns. They improve the basis for material translation; **visual fidelity still requires original-render comparisons, and source reuse requires resolving the stated terms.**